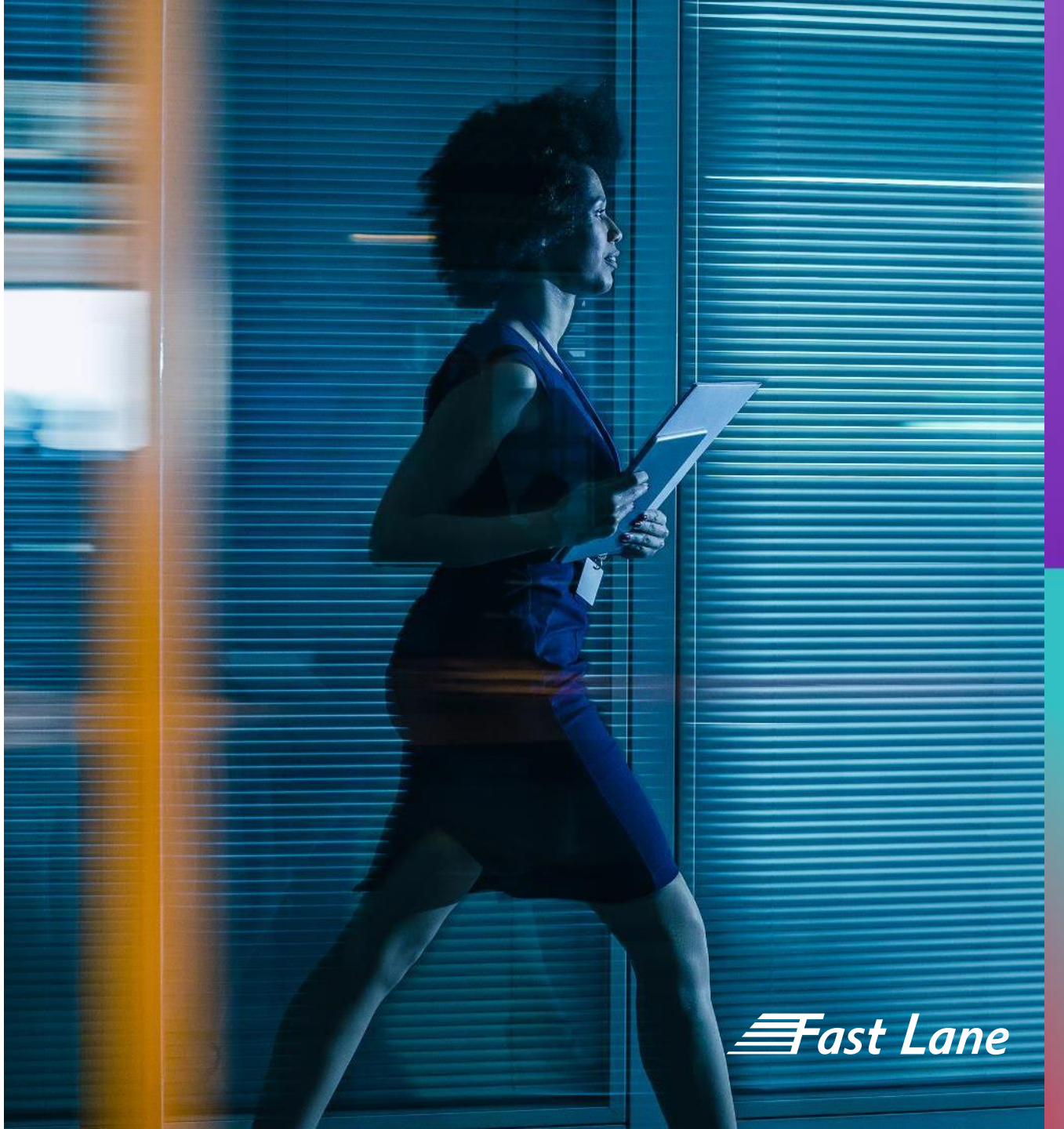
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 11

Planning & Spec-Driven Development

Analyze, plan and specify before any code

Agentic Software Engineering with Claude Code



01

Brownfield Analysis: Harvest the Current State

Processes, Excel, data and code into a gap list

02

Planning with Claude Code

Plan mode, a shared ledger, STATE.md and work packages

03

Why Spec-Driven Development

Why agents guess, four phases and the constitution

04

The Spec-Driven Workflow

Spec Kit artifacts, checks and the converge loop

05

Sample Case: Implement a Product Feature

A leave days preview through four checkpoints

Analysis, planning and specs

Written contracts Claude satisfies, checked phase by phase

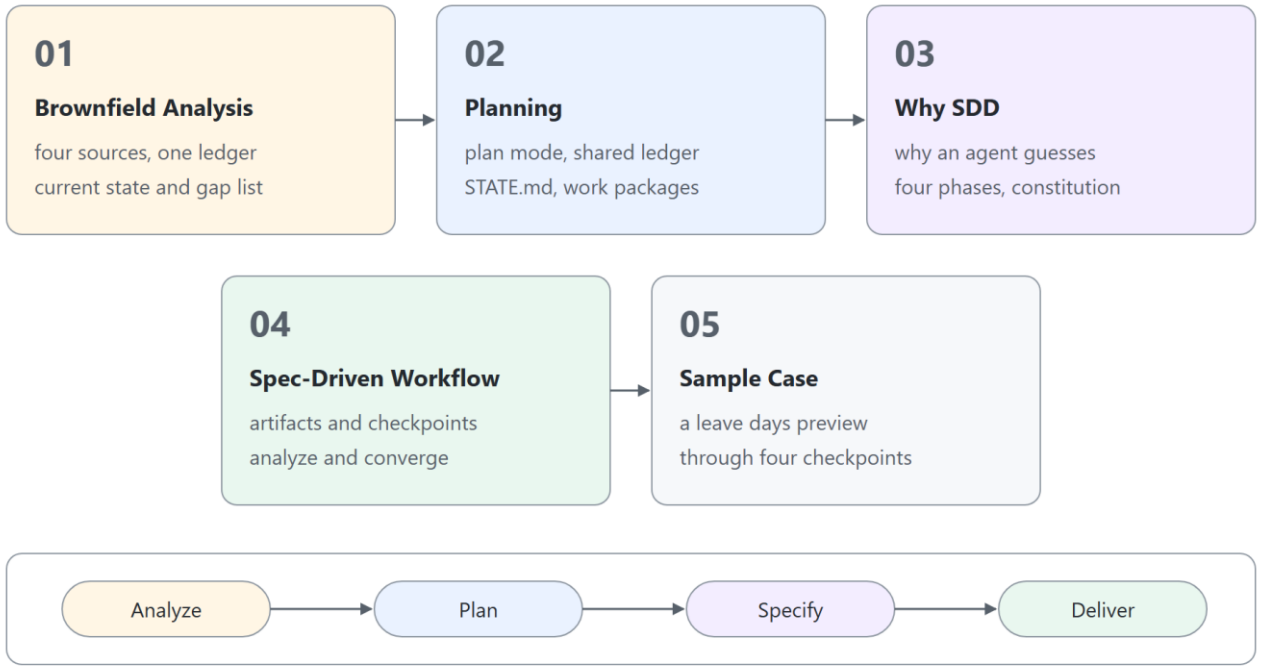
- **Analysis**: harvest the current state into one document with a gap list
- **Planning**: plan mode, a shared ledger, `STATE.md` and work packages
- **Spec-driven development**: constitution, spec, plan and tasks with GitHub Spec Kit



Overview

The path from what runs today to a checked spec

- **Analyze** the current state before anything is planned
- **Plan** in plan mode, research through a ledger, hand off in `STATE.md`
- Learn why a **specification** beats a longer prompt
- Run the **Spec Kit** pipeline and the commands that check it
- Take a real feature through all **four checkpoints**



01

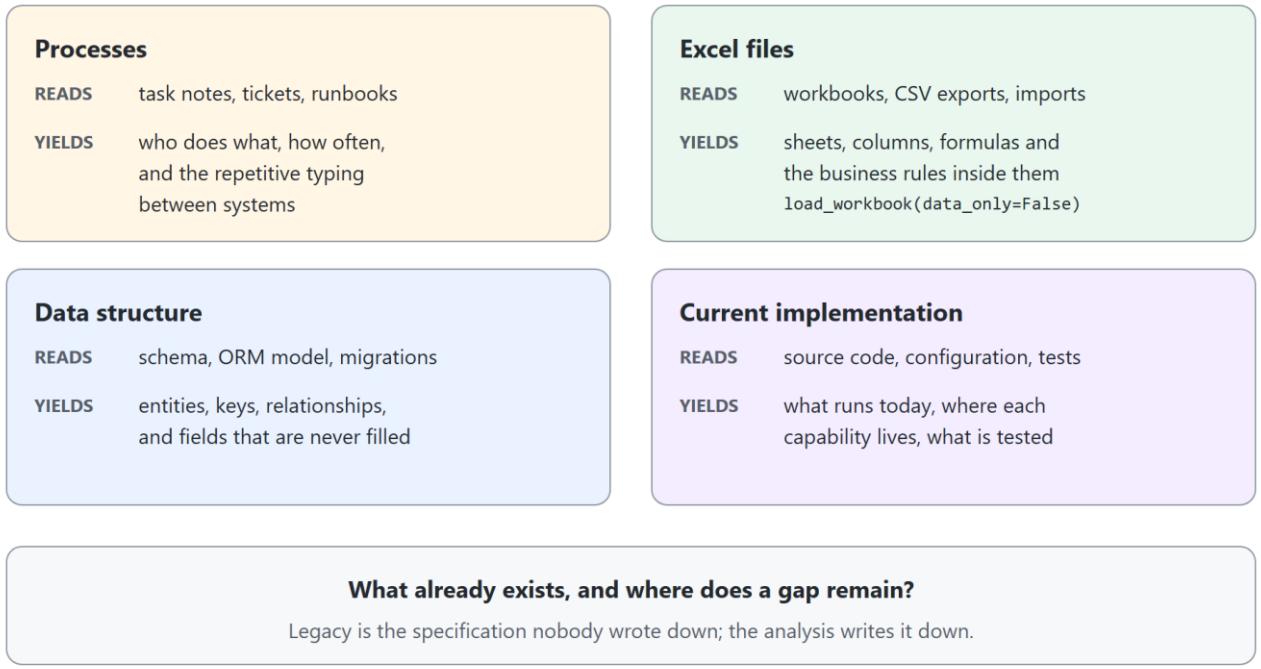
Brownfield Analysis: Harvest the Current State

Processes, Excel, data and code into a gap list

The four sources

Where the current state of a system lives

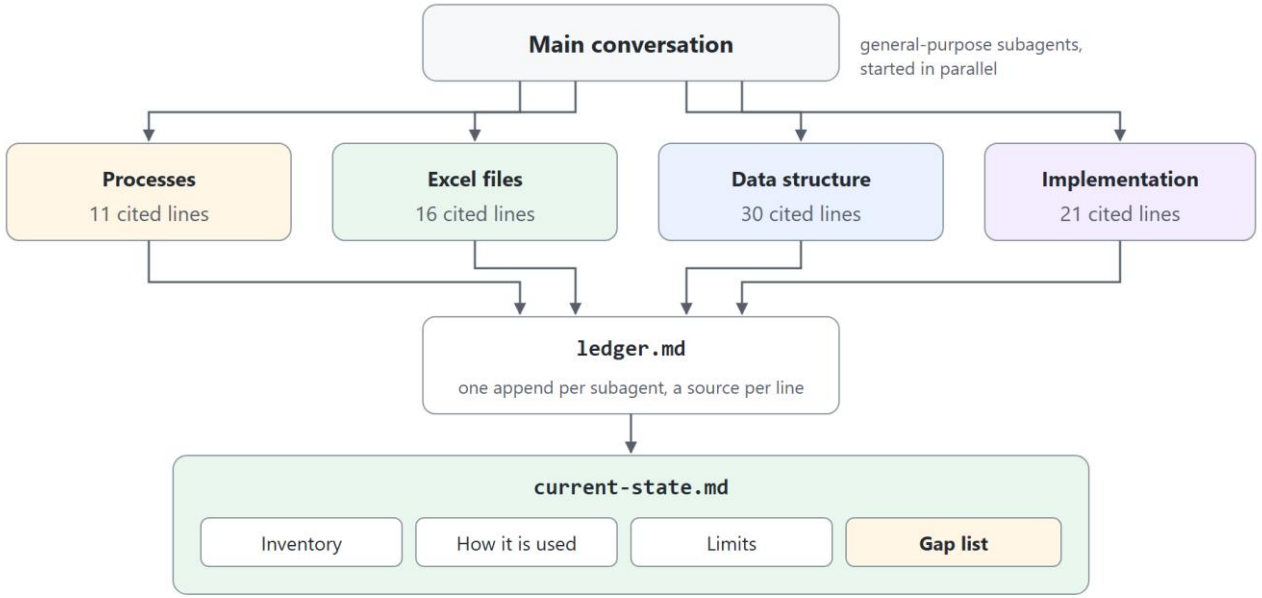
- **Processes**: task notes and runbooks show who does what, and how often
- **Excel files**: formulas hold business rules nobody wrote down
- **Data structure**: schema and model show fields that are never filled
- **Implementation**: code, config and tests show what runs and what is tested
- The guiding question: what exists, and where does a gap remain?



Harvesting into a ledger

Parallel subagents, one cited file and a current-state document

- One **subagent per source**, started in parallel from one prompt
- `general-purpose` subagents can write; `Explore` and `Plan` are read-only
- Each writes its section to `ledger.md` in a **single append**
- `current-state.md`: Inventory, How it is used, Limits, **Gap list**
- Check the document against the sources before anyone plans from it



Check every Limits line and Gap list row against the source it cites.

02

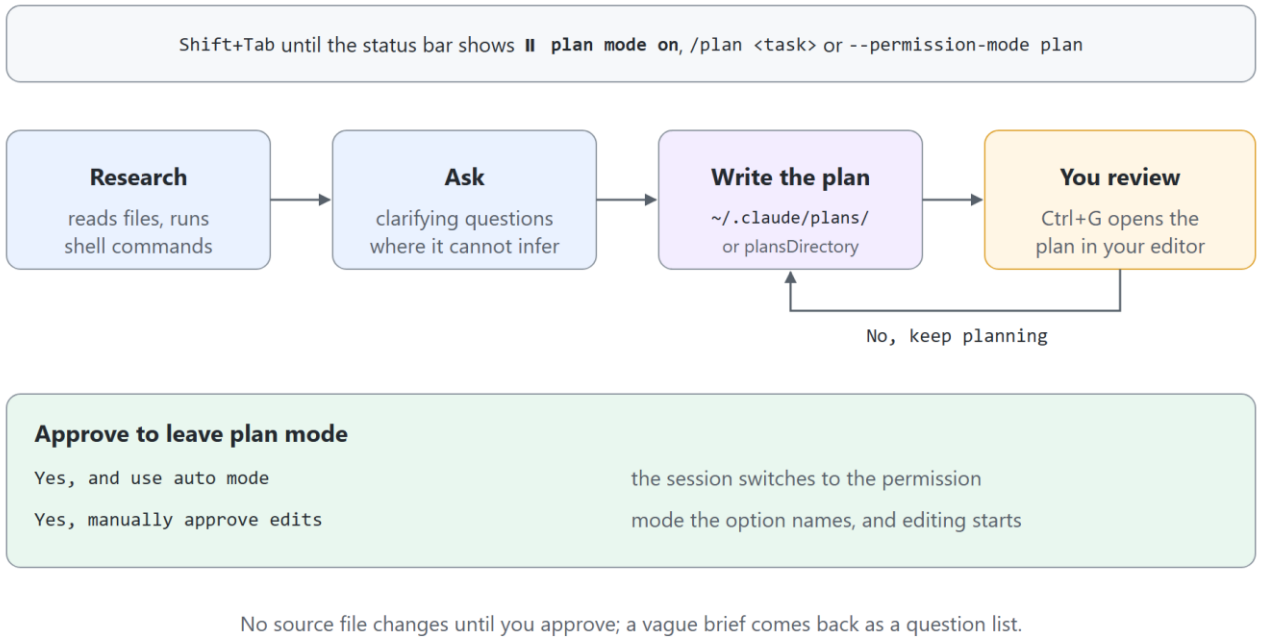
Planning with Claude Code

Plan mode, a shared ledger, STATE.md and work packages

Plan mode

Research and a reviewed plan before any edit

- `Shift+Tab` until the status bar shows **plan mode on**, or `/plan <task>`
- Claude researches, writes a **plan file**, and leaves your source untouched
- `Ctrl+G` opens the plan in your editor before you approve it
- A vague brief comes back as a **question list**, the first deliverable
- Skip the plan when you could describe the diff in one sentence

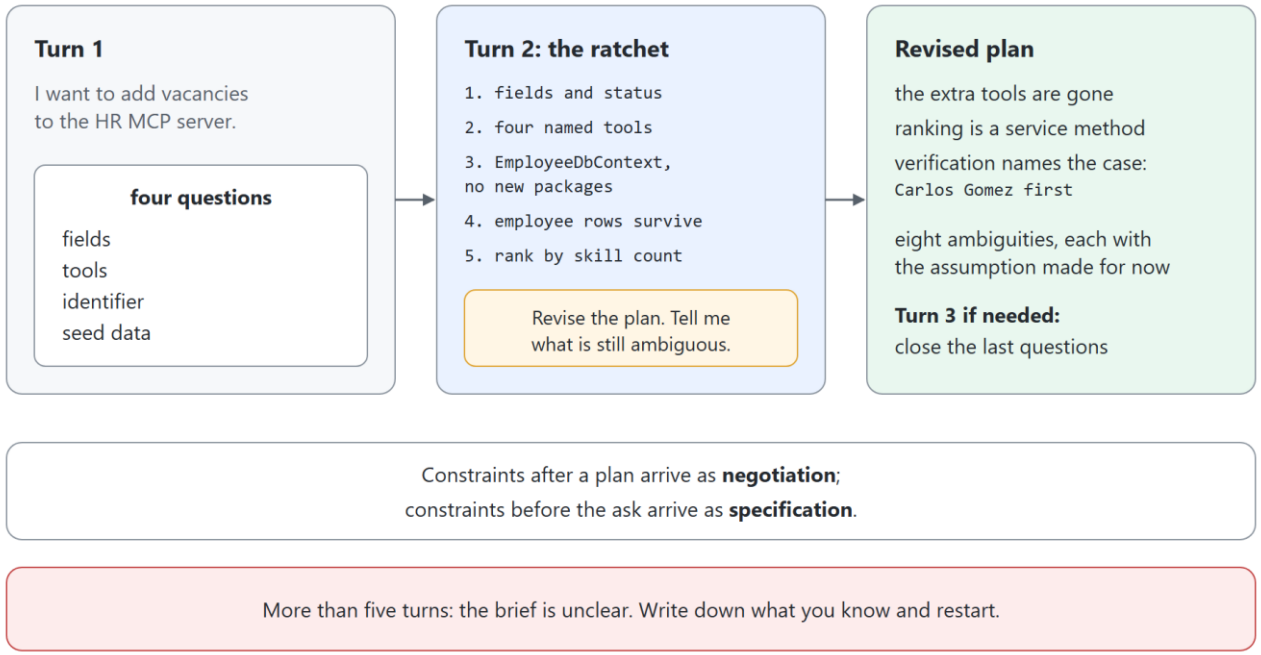


No source file changes until you approve; a vague brief comes back as a question list.

The constraint ratchet

Constraints before the ask are read as specification

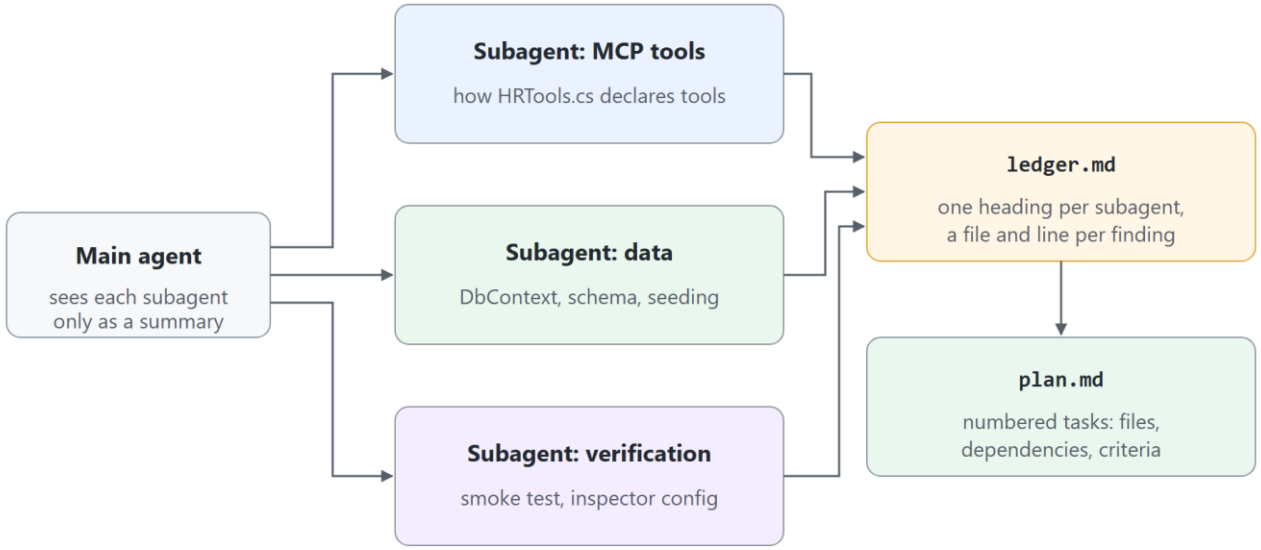
- Constraints after a plan arrive as **negotiation**, before the ask as **specification**
- A numbered block of non-negotiables locks the decisions already made
- End with **"Tell me what is still ambiguous"** so Claude names its gaps
- Two or three turns is normal; past five, rewrite the brief



Parallel research through a ledger

Subagents share their findings through one file

- Research is slow and mostly independent: one **subagent** per area
- A subagent sees none of your conversation, only its delegation message
- Every subagent appends under its own heading in `ledger.md`
- Claude writes `plan.md` from the ledger: two artifacts to review
- A plan is ready when someone who never saw the chat can execute it

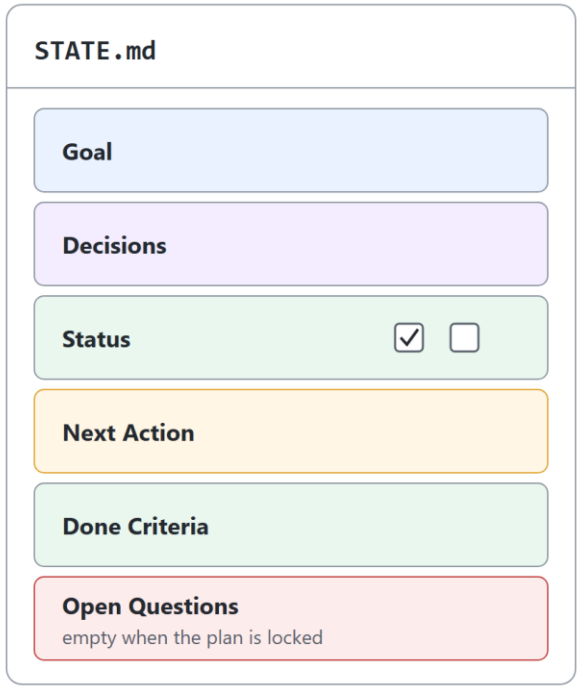


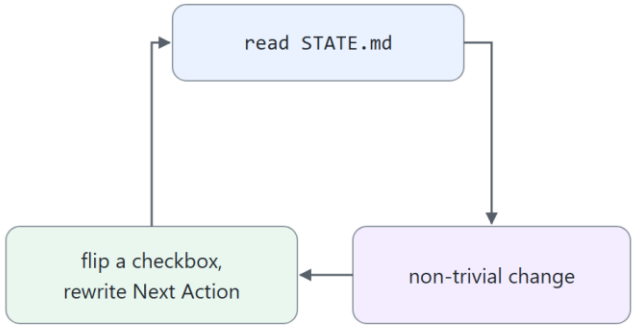
A doubtful plan claim traces to the ledger line it came from.

STATE.md as a rolling handoff

The current state, read and updated every session

- It records the **current** state; a spec describes the end state
- Sections: Goal, Decisions, Status, **Next Action**, Done Criteria, Open Questions
- The plan is locked when **Open Questions** is empty
- One `CLAUDE.md` rule: read it before, update it after every non-trivial change
- One skipped update starts the drift



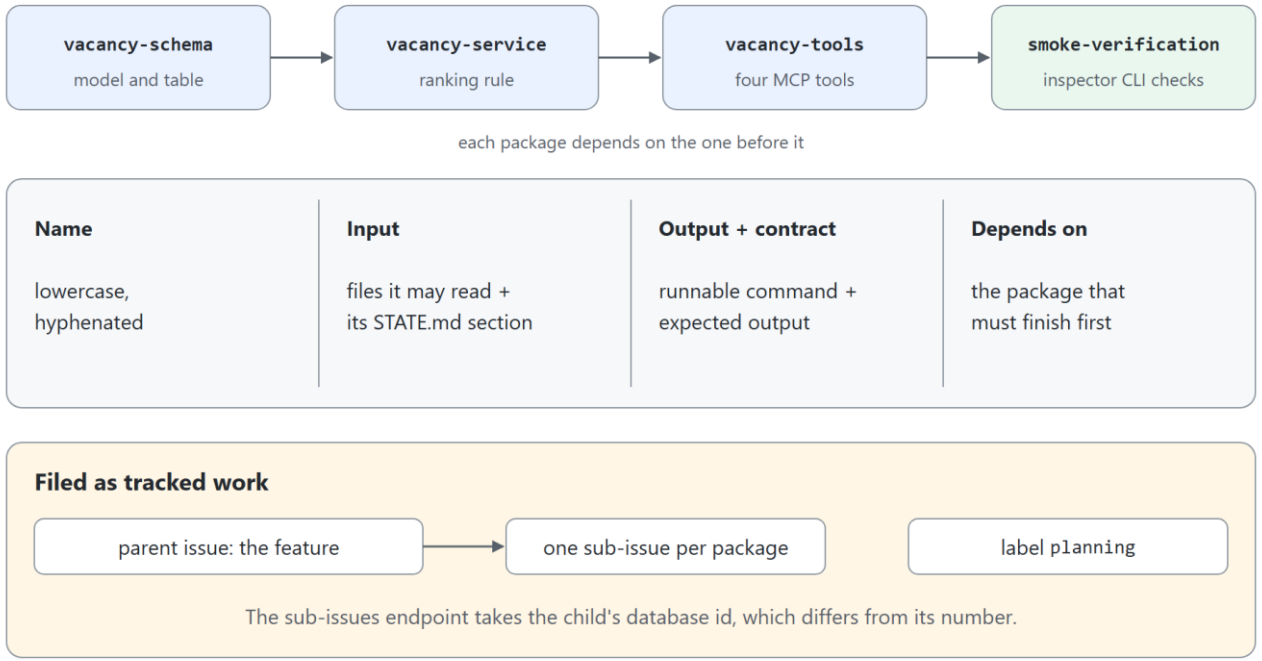



```
CLAUDE.md:
Read STATE.md before any
non-trivial change. Update it after.
```

Work packages with contracts

Units one session can finish without asking

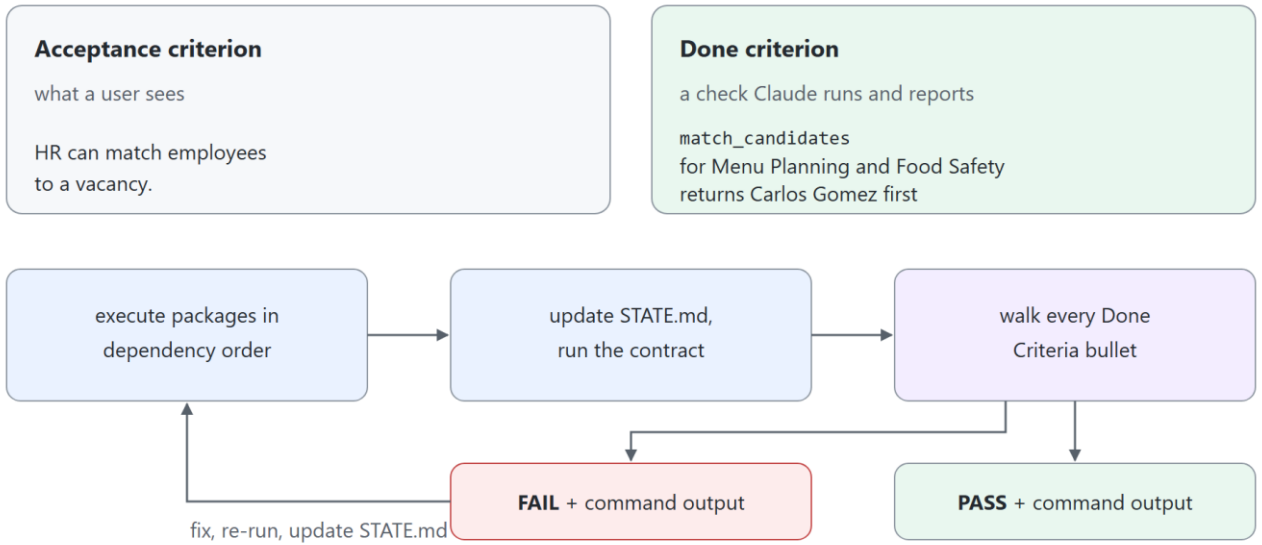
- Four fields: **Name**, **Input**, **Output** with its contract, **Depends on**
- The contract is a **runnable command** and its expected output
- One concern per package; three to five packages, more than seven is too fine
- Parallel packages declare no dependency on each other
- File the plan as a parent issue with one **sub-issue** per package



Done criteria and self-critique

Evidence for every criterion

- **Done criteria** are checks Claude can run and report unambiguously
- Acceptance criteria describe what a user sees; done criteria name a check
- After execution, Claude walks every bullet and reports **PASS** or **FAIL**
- Each result carries the actual command output
- The feature is complete when every bullet is PASS



If Claude has to interpret whether a criterion is met, it is not specific enough.

03

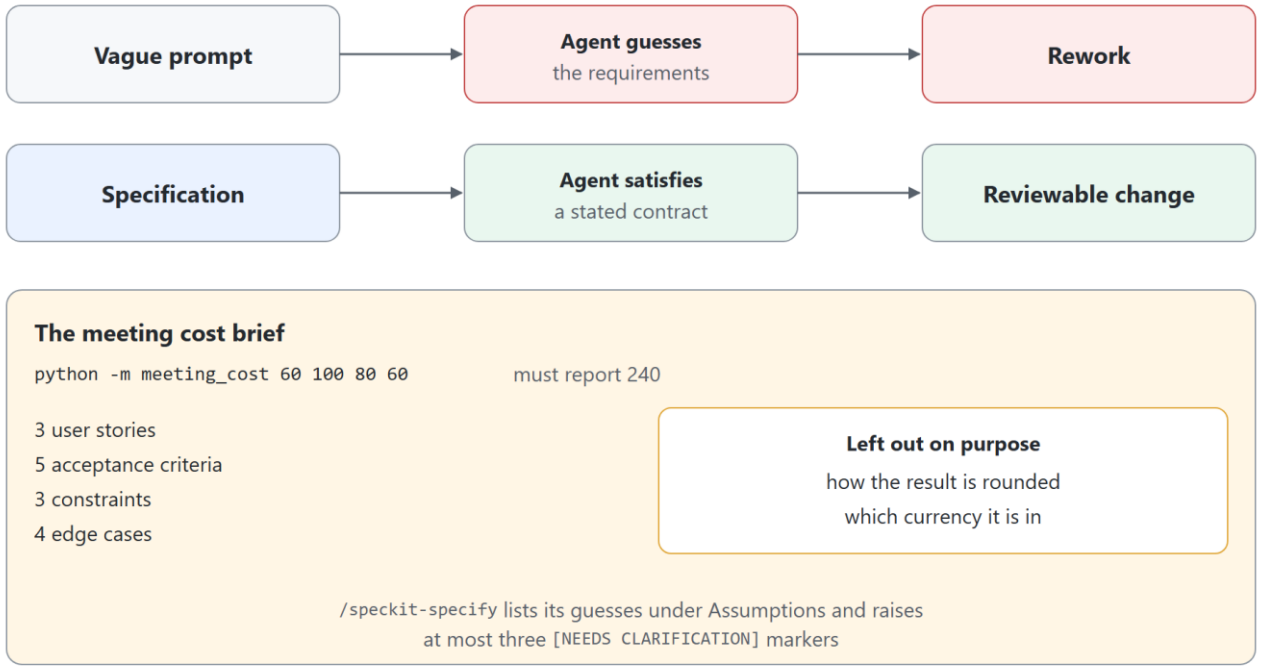
Why Spec-Driven Development

Why agents guess, four phases and the constitution

Why an agent guesses

A specification turns guesses into decisions you read

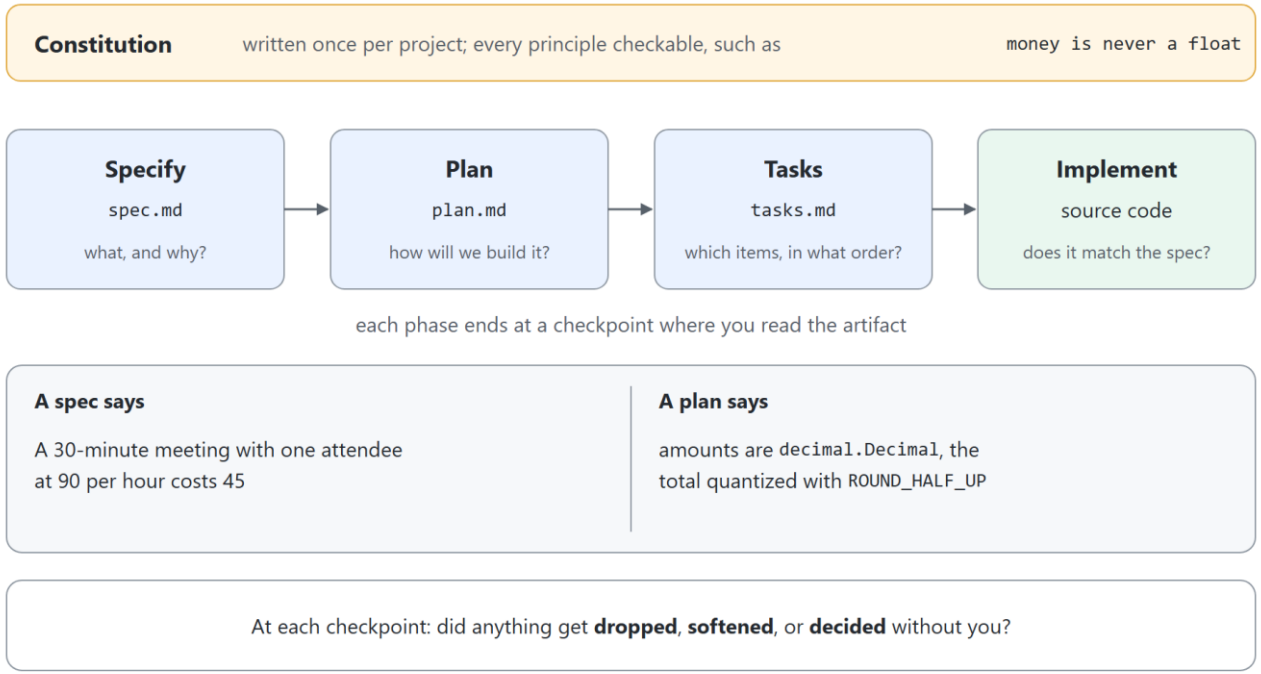
- Every request leaves out details that feel self-evident
- A developer asks; an agent picks an answer and **keeps moving**
- The meeting cost brief leaves out **rounding** and **currency** on purpose
- `/speckit-specify` lists its guesses under **Assumptions**
- At most three `[NEEDS CLARIFICATION]` markers, for the biggest decisions



Four phases and a constitution

Each phase ends at a checkpoint you read

- **Specify** `spec.md`, **Plan** `plan.md`, **Tasks** `tasks.md`, **Implement** the code
- A spec says what and why; a plan names types, modules and libraries
- The **constitution** holds checkable principles for every feature
- "Money is never a float" shows up where `plan.md` names a type
- At each checkpoint: dropped, softened, or decided without you?



When spec-driven development pays off

Four documents and four reviews before the feature is done

- A one-line fix or a throwaway script needs a direct prompt
- Reach for it when someone will hold you to the **acceptance criteria**
- Or when the request comes from someone other than the implementer
- Or when an agent will change the code later
- Install with `--reinstall`, initialize with `--integration claude`



04

The Spec-Driven Workflow

Spec Kit artifacts, checks and the converge loop

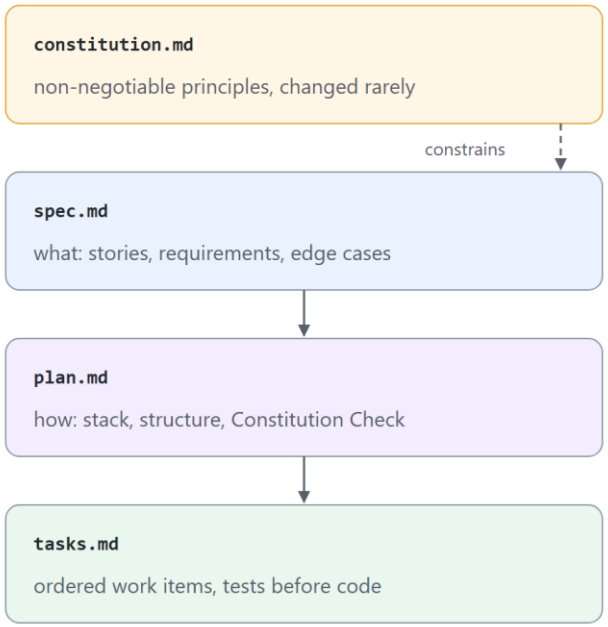
The four artifacts

Plain Markdown next to the code, reviewed in the pull request

- `constitution.md`: non-negotiable principles, changed rarely
- `spec.md`: user stories, requirements, edge cases and success criteria
- `plan.md` plus research, data model, **contracts** and quickstart
- `tasks.md`: numbered tasks in phases, `[P]` marks parallel work
- `specify init` writes `.specify/` and ten **skills** under `.claude/skills/`

```
my-project/
   .claude/skills/
      speckit-<cmd>/SKILL.md, ten skills
   .specify/
      memory/constitution.md
      scripts/
      templates/
   meeting_cost/
   specs/
      001-meeting-cost-calculator/
         spec.md
         plan.md
         research.md
         data-model.md
         contracts/
         quickstart.md
         tasks.md

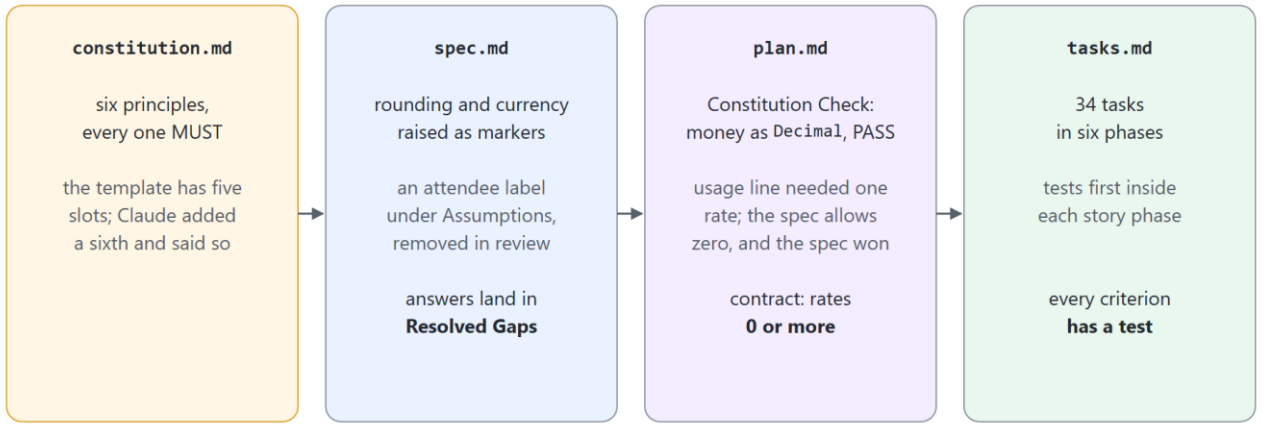
specify init writes all but specs/ and the package
```



What each checkpoint caught

The recorded meeting cost run, artifact by artifact

- Constitution: six principles kept as **MUST**, one more than the template
- Spec: rounding and currency raised as markers, an invented label removed
- Plan: the Constitution Check names `Decimal` for money
- Contract: the usage line needed a rate, the spec allowed zero; the spec won
- Tasks: 34 tasks in six phases, every criterion behind a test



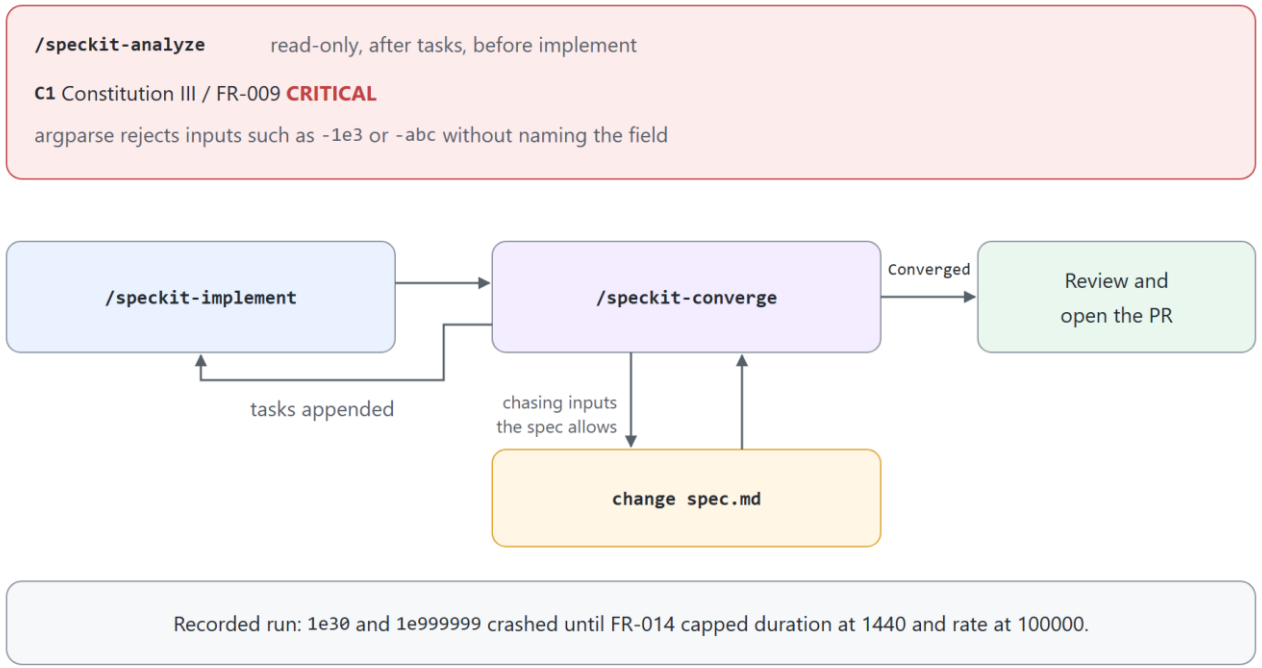
Resolved Gaps in the recorded spec.md

Rounding: the total is rounded half up to two decimal places; breakdown values stay unrounded.
Currency: amounts are currency-agnostic decimals with no symbol.
Attendees: a list of hourly rates, identified by position.

Analyze and converge

Commands that check the documents and the code

- `/speckit-clarify` asks up to five questions and writes the answers back
- `/speckit-analyze` is read-only; a constitution conflict is always **CRITICAL**
- It found a CRITICAL conflict on negative inputs before any code existed
- `/speckit-converge` appends unbuilt work to `tasks.md` until **Converged**
- When the loop keeps chasing inputs, change the **spec**



05

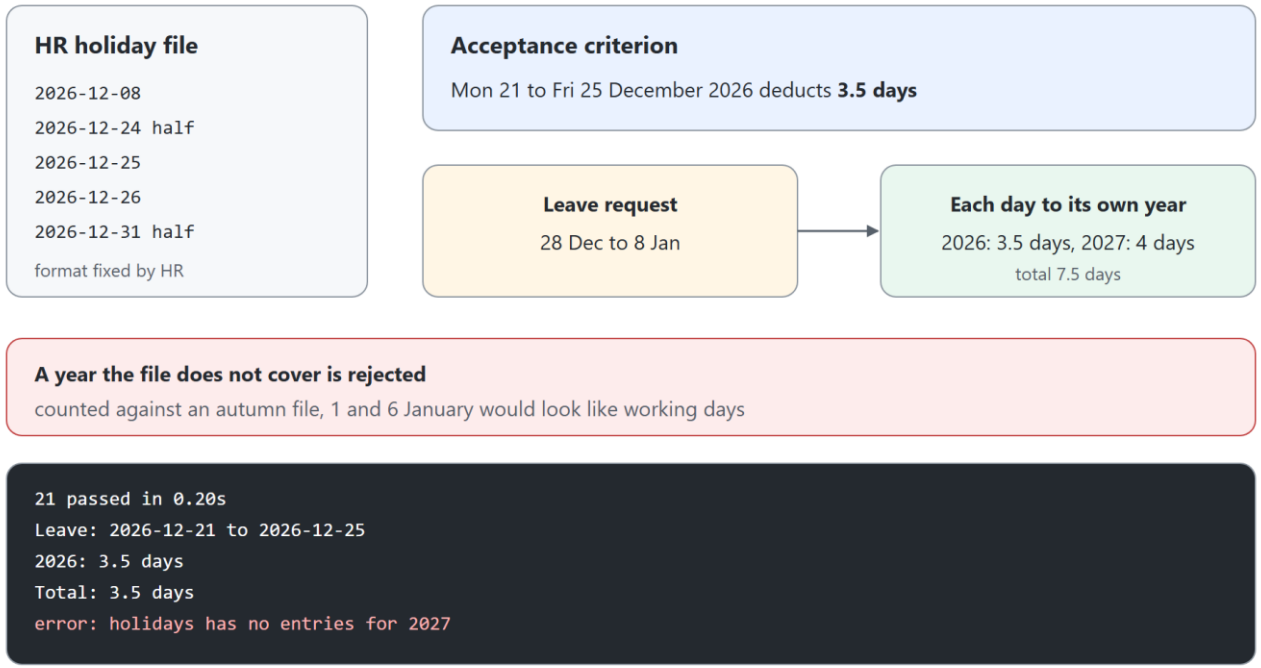
Sample Case: Implement a Product Feature

A leave days preview through four checkpoints

A leave days preview

A second feature through all four checkpoints

- Monday 21 to Friday 25 December 2026 deducts **3.5 days**
- Constitution: exact half days as `Decimal`, no holiday date in source code
- Spec: a request across New Year charges each day to **its own year**
- Spec: a year the holiday file does not cover is **rejected**
- `21 passed`, and a request beyond the holiday file is refused with an error



What the checkpoints bought

Each decision is a sentence written before the code

- Charge each day to its own year: decided at **specify**
- Reject a year the file does not cover: decided at **specify**
- Half days are exact decimals: **constitution and plan**
- Holidays come only from HR's file: **constitution and plan**
- Each answer was written by the person who owns it



Takeaways

Analyze, plan, specify, verify

- Harvest the **current state** with parallel subagents before you plan
- Put constraints **before** the ask, then ask what is still ambiguous
- Keep **STATE.md** current and give every package a runnable contract
- Read each artifact at its checkpoint: dropped, softened, or decided without you?
- Accept a feature when every criterion has evidence and converge reports **Converged**

